

กลุ่มที่ 3

Text Editor

Snapshot Method vs Command / Delta Method

การออกแบบและวิเคราะห์อัลกอริทึมด้วย Stack โดยใช้ภาษา Java

สมาชิกกลุ่ม:

นายเนติธร กาฬักดี นายนิศ สุขอยู่ นายพีรพัฒน์ ไคร์ในธรรม นางสาวทิพวรรณ พิมพ์มาตร

นายภูรินทร์ ภาววงษ์ นายภูมิภัทร ศรีสุวรรณ นางสาวกัญญาดา ใจบาน

Case Story

โปรแกรม Text Editor ที่รองรับ Undo / Redo

- โปรแกรม Text Editor ต้องรองรับการเพิ่ม (INSERT) ลบ (DELETE) และแทนที่ (REPLACE) ข้อความ
- ผู้ใช้สามารถ Undo และ Redo การกระทำได้
- กติกาสำคัญ: เมื่อผู้ใช้ Undo แล้วทำ Action ใหม่ ประวัติใน Redo Stack ต้องถูกล้างทิ้งทันที
- Action ที่ต้องรองรับมีข้อมูล: Action ID, Action Type, Position, Old Text, New Text, Timestamp

ตัวอย่าง

เดิม: HelloWorld

INSERT "AI" ที่ตำแหน่ง 5

ผลลัพธ์: HelloAIWorld

Undo → กลับเป็น HelloWorld

Redo → กลับเป็น HelloAIWorld

Input, Output และ Constraints

Input

- ข้อความเริ่มต้น (อาจว่าง)
- INSERT(position, text)
- DELETE(position, length)
- REPLACE(position, length, newText)
- UNDO / REDO

Output

- ข้อความปัจจุบันหลังแต่ละคำสั่ง
- สถานะ Undo/Redo Stack
- ผลสำเร็จ/ล้มเหลวของคำสั่ง

Constraints

- $0 \leq \text{position} \leq \text{length}(\text{document})$
- $\text{position} + \text{length} \leq \text{length}(\text{document})$
- $\text{length} \geq 0$ เสมอ
- Redo Stack ต้องถูกล้างเมื่อมี Action ใหม่หลัง Undo

Algorithm A: Snapshot Method

แนวคิดหลัก

บันทึกสำเนาเอกสารทั้งฉบับลง Stack ก่อนทุกการแก้ไข การ Undo/Redo คือการสลับสำเนาระหว่าง undoStack และ redoStack

- ข้อดี: เข้าใจง่าย พิสูจน์ถูกต้องง่าย รองรับทุกประเภทการแก้ไขทันที
- ข้อจำกัด: ใช้หน่วยความจำสูงมาก (คัดลอกทั้งเอกสารทุกครั้ง)
- Time Complexity: $O(n)$ ต่อการกระทำหนึ่งครั้ง
- Space Complexity: $O(n \times m)$

Pseudocode (ย่อ)

```
1. undoStack.push(copy of document)
2. IF redoStack not empty
3.     Clear redoStack
4. document <- apply edit
5. RETURN document
```

UNDO:

```
1. IF undoStack empty: RETURN false
2. redoStack.push(copy of document)
3. document <- undoStack.pop()
```

Algorithm B: Command / Delta Method

แนวคิดหลัก

เก็บเฉพาะ Action object (type, position, oldText, newText) แล้วคำนวณ Inverse Operation เพื่อ Undo แทนการเก็บเอกสารทั้งฉบับ

- ข้อดี: ประหยัดหน่วยความจำมาก เร็วกว่าเมื่อเอกสารมีขนาดใหญ่
- ข้อจำกัด: ต้องออกแบบ Inverse Operation ให้ถูกต้องทุกกรณี ซับซ้อนกว่า
- Time Complexity: $O(k)$ โดย k = ความยาวข้อความที่เปลี่ยนแปลง
- Space Complexity: $O(k \times m)$

Inverse Operation

```
INSERT(pos, text)
    inverse = DELETE(pos, len(text))
```

```
DELETE(pos, removedText)
    inverse = INSERT(pos, removedText)
```

```
REPLACE(pos, oldText, newText)
    inverse = REPLACE(pos, newText, oldText)
```

ตัวอย่าง: INSERT "AI" ที่ตำแหน่ง 5

inverse -> DELETE ตำแหน่ง 5 ยาว 2 ตัวอักษร

ตัวอย่าง Step-by-step (Algorithm B)

ลำดับ	คำสั่ง	เอกสารหลังทำ	Undo Stack (บนสุด)
1	INSERT(0,"HelloWorld")	HelloWorld	I(0,"HelloWorld")
2	INSERT(5,"AI")	HelloAIWorld	I(5,"AI")
3	DELETE(0,5)	AIWorld	D(0,"Hello")
4	UNDO	HelloAIWorld	I(5,"AI")
5	UNDO	HelloWorld	I(0,"HelloWorld")
6	REDO	HelloAIWorld	I(5,"AI")

จุดสังเกต

- ทุก Action ถูก push ก่อน แล้วจึง apply
- Undo = pop + apply inverse + push ไปยัง redoStack
- Redo = pop จาก redoStack + apply forward
- ลำดับข้อความส่วนที่ไม่เกี่ยวข้องไม่เปลี่ยน

Correctness Argument

Precondition / Invariant / Postcondition — Algorithm B

1

Precondition

document คือข้อความปัจจุบัน
ที่ถูกต้อง
undoStack/redoStack
สอดคล้องกับประวัติจริง

2

Loop Invariant

ก่อนแต่ละรอบของ Undo ผล
จาก Action ทั้งหมดใน
undoStack เมื่อ apply กับ
เอกสารเริ่มต้น ต้องเท่ากับ
document ปัจจุบันเสมอ

3

Maintenance (Proof by Cases)

INSERT/DELETE/REPLACE แต่
ละแบบมี Inverse ที่คืนค่าตรง
ข้ามได้แม่นยำ — พิสูจน์แยกที
ละกรณี

4

Termination

ทุก Operation เป็นแบบไม่เรียก
ซ้ำ จำนวนขั้นตอนจำกัดเสมอ
จึงหยุดทำงานได้เสมอ

5

Postcondition

document ตรงกับผลรวมของ
Action ที่เหลือใน undoStack;
ข้อมูลที่ไม่เกี่ยวข้องไม่สูญหาย
หรือสลับลำดับ

Big-O Analysis

n = ความยาวเอกสาร, k = ความยาวข้อความที่เปลี่ยน, m = จำนวน Action สะสม

Algorithm	Best Case	Average Case	Worst Case	Auxiliary Space
A: Snapshot	$O(1)$	$O(n)$	$O(n)$	$O(n \times m)$
B: Command/Delta	$O(1)$	$O(k)$	$O(n)$	$O(k \times m)$

- $\text{push()}/\text{pop()}$ ของ Stack เองเป็น $O(1)$ เสมอ แต่ต้นทุนหลักอยู่ที่การคัดลอก/ย้ายข้อความ ไม่ใช่ตัว Stack
- Algorithm A: ทุก Insert/Delete/Replace/Undo/Redo คัดลอกเอกสารทั้งฉบับ $\rightarrow O(n)$ เสมอ
- Algorithm B: ต้นทุนขึ้นกับขนาดของการแก้ไขจริง (k) เท่านั้น ไม่ใช่ทั้งเอกสาร
- เมื่อ $k \ll n$ (กรณีทั่วไป) Algorithm B ประหยัดทั้งเวลาและหน่วยความจำกว่ามาก

Class Diagram และ Java Implementation

```
<<interface>> TextEditor
insert/delete/replace
undo() / redo()
getPushCount/PopCount/...

      ^           ^
      |implements |implements
SnapshotEditor  CommandEditor
(Algorithm A)   (Algorithm B)

Action
actionId, actionType
position, oldText, newText
timestamp

Main
menu loop + validation
selects Algorithm A / B

PerformanceExperiment
timed trials -> CSV export
```

- Input Validation: ตรวจสอบขอบเขต position/length ก่อนแก้ไขทุกครั้ง (IllegalArgumentException)
- การตรวจสอบ Stack ว่าง: undo()/redo() คืนค่า boolean แทนการโยน Exception
- การนับ Operation: pushCount, popCount, comparisonCount, auxiliaryCharCount ในทุกคลาส
- Error Handling: Main.java ครอบหุ้ทุกคำสั่งด้วย try-catch แสดงข้อความแจ้งเตือนโดยไม่ crash

Test Cases

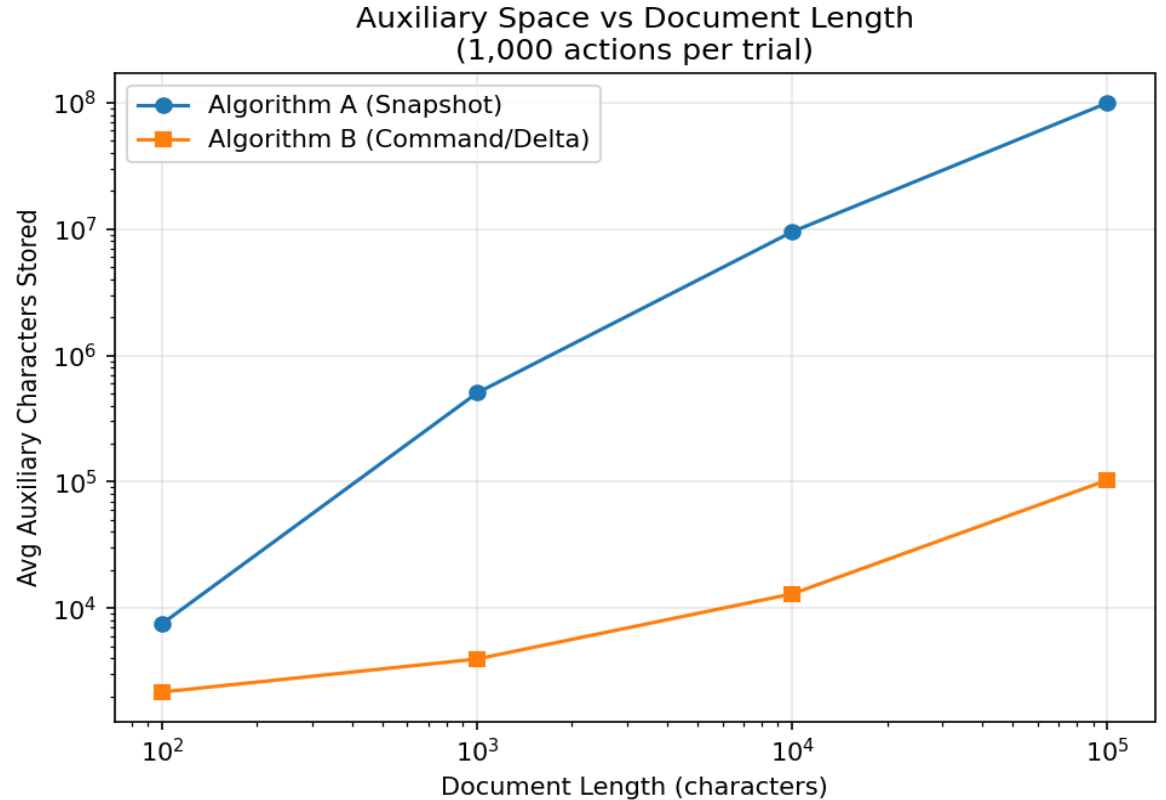
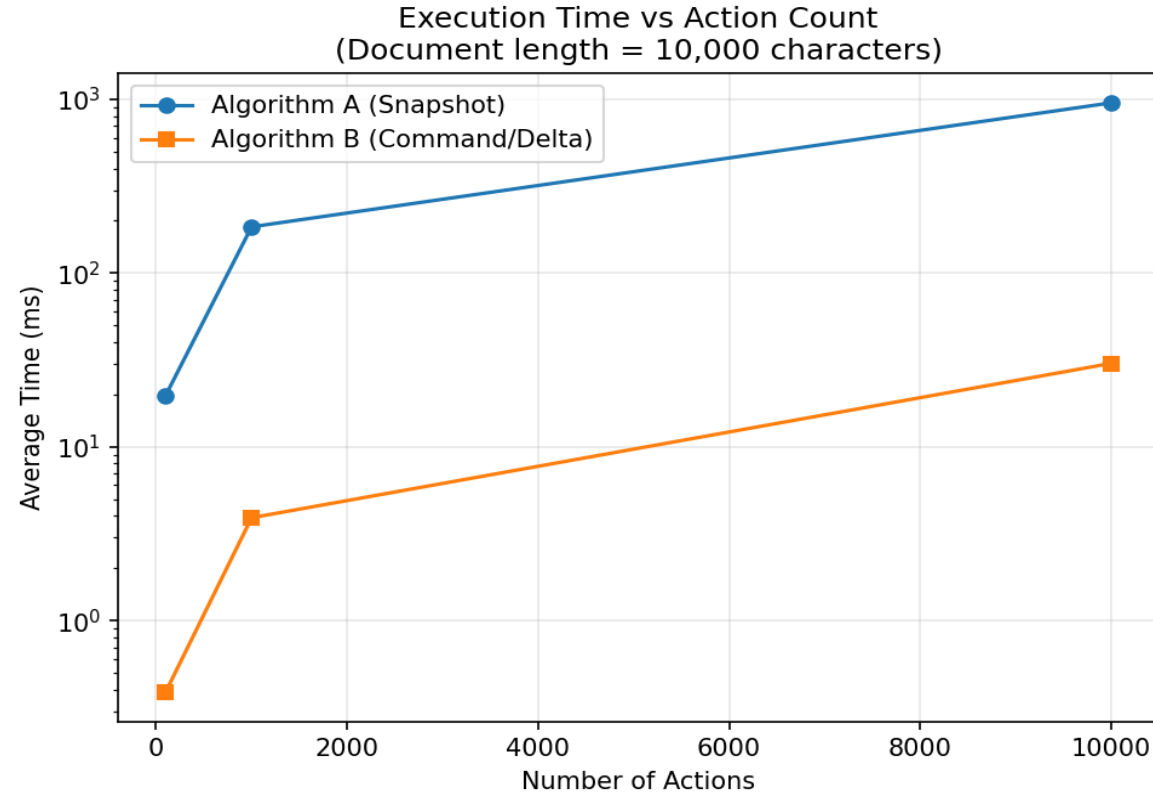
18 กรณีทดสอบ ครอบคลุมทุกหมวดที่กำหนด

Test ID	หมวด	Input / Action	Expected
TC01	Normal	INSERT "AI" ที่ตำแหน่ง 5	"HelloAIWorld"
TC08	Boundary	UNDO จนกว่า Stack ว่าง	แจ้งเตือนเมื่อว่าง ไม่ crash
TC11	Redo Invalidation	UNDO แล้วทำ Action ใหม่	Redo Stack ถูกล้าง
TC12	Invalid Input	INSERT ตำแหน่งเกินความยาว	ปฏิเสธ พร้อม validation error
TC14	Empty Input	DELETE บนเอกสารว่าง	แจ้งเตือน ไม่ crash
TC18	Algorithm Comparison	รันชุดคำสั่งเดียวกันบน A และ B	ผลลัพธ์เอกสารตรงกันทุกประการ

รายการทดสอบฉบับเต็ม 18 กรณี อยู่ในไฟล์ *Stack_Group03_TestCases.xlsx*

Experimental Results

ทดสอบ 4 ขนาดเอกสาร × 3 จำนวน Action × 5 รอบ



- เอกสาร 100,000 ตัวอักษร / 10,000 Action: Algorithm A ≈ 20.98 วินาที เทียบกับ Algorithm B ≈ 0.25 วินาที (เร็วกว่า ~85 เท่า)
- ผลตรงกับ Big-O ที่วิเคราะห์ไว้ — Algorithm B ขยายตัวช้ากว่าทั้งด้านเวลาและหน่วยความจำอย่างชัดเจน

สรุปและวิธีที่กลุ่มเลือก

เลือก: Algorithm B (Command / Delta)

- เร็วกว่าและประหยัดหน่วยความจำกว่ามากในเอกสารขนาดใหญ่
- เหมาะกับพฤติกรรมพิมพ์จริงที่แก้ไขทีละเล็กน้อย ($k \ll n$)
- ผลทดลองสอดคล้องกับทฤษฎี Big-O อย่างชัดเจน

ข้อจำกัด และแนวทางต่อยอด

- B พัฒนา/ทดสอบยากกว่า เสี่ยงบั๊กจาก Inverse Operation ที่ผิดพลาด
- แนวทางต่อยอด: จำกัดขนาด Stack ด้วย LRU, รวม Action ที่พิมพ์ต่อเนื่องให้เป็นก้อนเดียว
- ขยายรองรับ Rich Text และ Multi-user Editing ในอนาคต